

1. User Upload Stage

When a user uploads a file:

- Accept only: **.pdf, .doc, .docx**
- Reject files larger than **5MB**
- Reject files over **5 pages**
- Reject images and unsupported formats
- Save the uploaded file to Supabase storage under:
uploads/{user_id}/{filename}

Insert a database row with the following exact fields:

original_filename

file_path

file_type

file_size

analysis_result = null

suggestions = null

score = null

structured_data = null

analyzed_at = null

extracted_cv = null

Never modify or rename DB fields.

2. File Parsing Stage

Once uploaded:

2.1 If PDF

- Try extracting text using **pypdf**
- If text extraction is empty or fails:
 - Convert pages to images (pdf2image)

- Use **pytesseract OCR** on each page

2.2 If DOC / DOCX

- Use python-docx or text extraction via docx2txt
- Convert embedded images to OCR if needed

2.3 After extraction

Produce a JSON object:

```
{  
  "raw_text": "...",  
  "is_scanned": true/false,  
  "page_count": 1-5  
}
```

Save this into **extracted_cv** field.

Only save valid JSON as a string.

3. Local NLP Analysis Stage (Open Source Only)

Use:

- spaCy
- textstat
- resume-parser

Generate **analysis_result**:

```
{  
  "readability_score": number,  
  "grade_level": "string",  
  "detected_entities": {  
    "names": [],  
    "emails": [],  
    "companies": [],
```

```
"dates": []  
},  
"length_in_tokens": number,  
"weak_sections": ["summary too short", "missing skills", ...]  
}
```

Generate **score** with a 0–100 formula:

readability * 1.2

+ number_of_skills * 3

+ number_of_experiences * 4

Clamp score between **0 and 100**.

4. Structured Data Stage

Run resume-parser (or custom regex rules) to generate structured CV data:

```
{  
  "personal_info": {  
    "name": "",  
    "email": "",  
    "phone": "",  
    "location": ""  
  },  
  "summary": "",  
  "experience": [],  
  "education": [],  
  "skills": [],  
  "certifications": [],  
  "languages": [],  
  "projects": []  
}
```

Save this into the **structured_data** field (as JSON string).

5. Groq Review Stage (AI Chat Review)

Send to Groq:

SYSTEM MESSAGE to Groq model:

You are a senior CV reviewer and hiring manager.

Your job is to review the user's extracted CV and structured_data.

Give precise and actionable improvements.

Rules:

- Do not hallucinate experience not present in the CV.
- Suggest improvements only.
- Make the language professional and concise.
- Never rewrite the entire CV unless the user asks.

USER MESSAGE to Groq:

Here is the raw extracted CV text:

{extracted_cv}

Here is the structured CV data:

{structured_data}

Here is the automatic analysis:

{analysis_result}

Provide:

1. Key weaknesses
2. What to improve
3. Missing elements
4. Specific changes section-by-section

6. Saving AI Suggestions

Save Groq output into:

- suggestions: full review from the AI
- updated_at: NOW()

Leave all other fields intact.

7. Final Step: Generate Updated CV Version

If the user clicks **"Generate Improved CV"**, send this final Groq prompt:

SYSTEM MESSAGE:

Rewrite the user's CV using professional language, correct structure, modern formatting, and clear bullet points.

Keep all factual experience. Do not invent anything.

USER MESSAGE:

Rewrite my CV based on this structured data:

{structured_data}

And apply the following improvements:

{suggestions}

Return a clean, ready-to-export CV.

Store it in a new field or new row (your choice).